

Title: Development and Implementation of a Website Blocker using Python and Tkinter

1. Introduction

The growing accessibility of the internet has made distractions a common issue, especially among students and professionals. To maintain productivity and ensure focus, controlling access to non-essential websites during work hours is crucial. This report outlines the development of a simple, user-friendly website blocker application using Python and its standard libraries, particularly Tkinter for the graphical user interface (GUI).

2. Objective

The main objective of this project is to design a system that can block access to specified websites during predefined hours (e.g., from 9 AM to 5 PM). The program is intended to be straightforward, customizable, and run locally on the user's machine.

3. Tools and Technologies Used

Programming Language: Python 3.13

Libraries:

Tkinter (for GUI)

threading (for running background tasks)

datetime (for managing and comparing current time)

time (for sleeping intervals)

4. System Requirements

Operating System: Windows 10 or later

Python version: 3.13 or compatible

Administrator privileges (required to modify the hosts file)

5. Program Design

The application has two primary components:

GUI for user input (Tkinter based)

Background thread for blocking logic

6. GUI Components The GUI consists of:

An entry field for adding website URLs

A list box to display all websites to be blocked

Buttons for adding/removing websites

Buttons to start and stop the blocking mechanism

7. Core Logic

The program manipulates the hosts file located at C:\Windows\System32\drivers\etc\hosts. By redirecting the domain names of the specified websites to 127.0.0.1, it effectively blocks them during the defined time period.

8. Code Implementation

```
import tkinter as tk
from tkinter import messagebox
from datetime import datetime
import threading
import time

hosts_path =
r"C:\Windows\System32\drivers\etc\hosts"
redirect = "127.0.0.1"
websites_to_block = []
running = False

def add_website():
    website = website_entry.get().strip()
    if website:
        websites_to_block.append(website)
        website_listbox.insert(tk.END, website)
        website_entry.delete(0, tk.END)

def remove_website():
    selected = website_listbox.curselection()
    if selected:
        index = selected[0]

websites_to_block.remove(website_listbox.get(index))
website_listbox.delete(index)
```

```

def blocker():
    global running
    while running:
        now = datetime.now()
        if 9 <= now.hour < 17:
            with open(hosts_path, 'r+') as file:
                content = file.read()
                for website in websites_to_block:
                    if website not in content:
                        file.write(f'{redirect} {website}\n')
        else:
            with open(hosts_path, 'r+') as file:
                content = file.readlines()
                file.seek(0)
                for line in content:
                    if not any(website in line for website in
websites_to_block):
                        file.write(line)
                file.truncate()
            time.sleep(5)

```

```

def start_blocker():
    global running
    if not running:
        running = True
        t = threading.Thread(target=blocker)
        t.start()
        messagebox.showinfo("Started", "Website
blocker started.")

```

```
def stop_blocker():
    global running
    running = False
    messagebox.showinfo("Stopped", "Website
blocker stopped.")

root = tk.Tk()
root.title("Website Blocker")

frame = tk.Frame(root)
frame.pack(pady=10)

website_entry = tk.Entry(frame, width=40)
website_entry.grid(row=0, column=0, padx=5)

add_button = tk.Button(frame, text="Add Website",
command=add_website)
add_button.grid(row=0, column=1, padx=5)

remove_button = tk.Button(frame, text="Remove
Selected", command=remove_website)
remove_button.grid(row=1, column=1, pady=5)

website_listbox = tk.Listbox(root, width=50)
website_listbox.pack(pady=10)

start_button = tk.Button(root, text="Start Blocking",
command=start_blocker, bg="green", fg="white")
start_button.pack(pady=5)
```

```
stop_button = tk.Button(root, text="Stop Blocking",  
command=stop_blocker, bg="red", fg="white")  
stop_button.pack(pady=5)
```

```
root.mainloop()
```

8. Flowchart Description

A flowchart visually represents the logic of the website blocker. The primary steps include:

STEP1:User inputs website(s)

STEP2:System stores the list

STEP3:On clicking "Start", the background thread begins

STEP4:Checks current time

STEP5:If within block time, writes entries to hosts file

STEP6:If outside block time, removes the entries

STEP7:Repeat until stopped

10. Error Handling

One major issue is `PermissionError`, which occurs when trying to write to the hosts file without administrator rights. This can be resolved by:

Running the script as an administrator

Modifying permissions on the hosts file (not recommended for security reasons)

11. Security Considerations

Always ensure your script is safe from unauthorized access

Avoid making your hosts file world-writable

12. Limitations

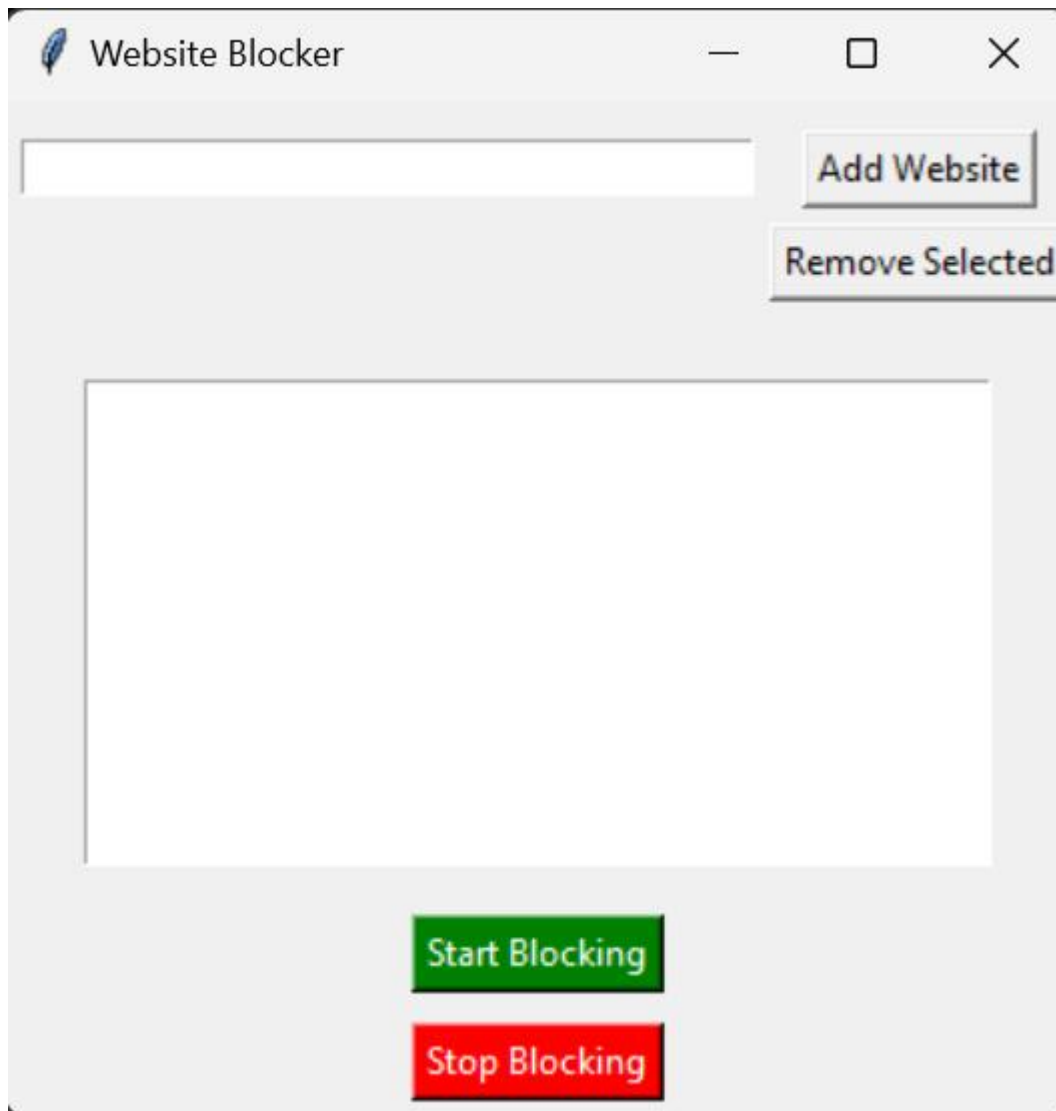
Only works on the local system

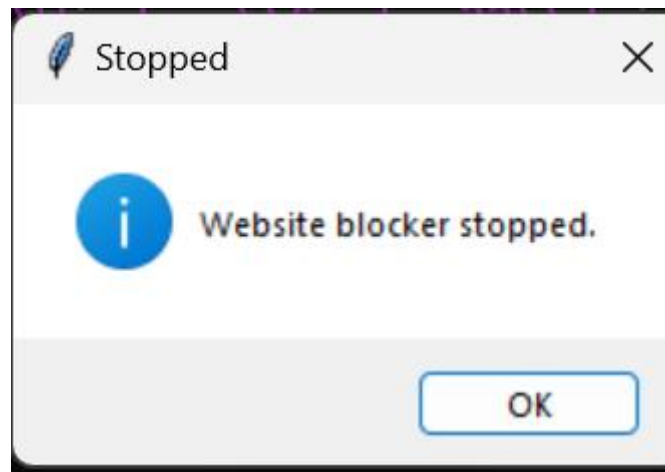
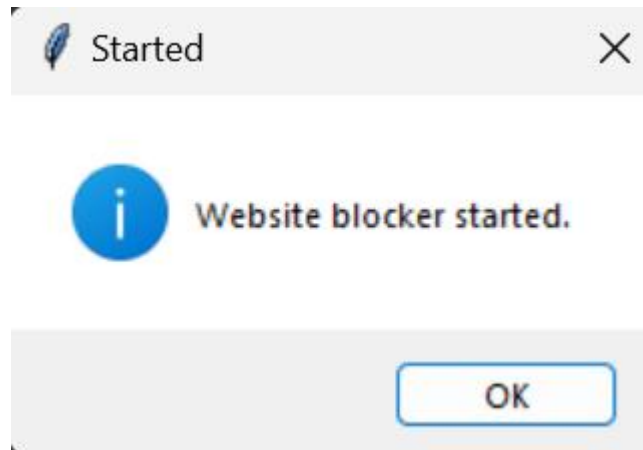
Users with admin rights can terminate the program

Doesn't block websites in HTTPS-only environments at the DNS level

Result:

GUI:





13. Conclusion

This project successfully implements a functional and extendable website blocker using Python. Its simplicity makes it ideal for students and individuals needing focus time. Future enhancements may include:

User-defined block schedules

Password protection for stopping the blocker

Logging and analytics features

14. Future Scope

Add support for macOS/Linux

Auto-run on boot

Integration with productivity tracking tools

15. References

Python Documentation:

<https://docs.python.org>

Tkinter GUI Guide: <https://tkdocs.com/>

Threading Module:

<https://docs.python.org/3/library/threading.html>
1

Hosts File:

[https://en.wikipedia.org/wiki/Hosts_\(file\)](https://en.wikipedia.org/wiki/Hosts_(file))